Introduction to Enterprise Application Development Using Spring Boot

Building Scalable, Robust, and Maintainable Modern Enterprise Applications

Learning Outcomes

- Understand Enterprise Applications & its Multi-Layer Design
- Configure and bootstrap First Spring Boot application
- Build your first REST API endpoint
- Integrate Swagger UI to test APIs
- Version control from Day One - Git and GitHub

Think About It: When millions of users buy products simultaneously on Amazon or transfer money on a banking app, why can't a basic Java app handle it?

Enterprise apps require strict security, multi-user concurrency handling, database connection handling, and seamless scalability!

What is an Enterprise Application?

An **Enterprise Application** is a large-scale software system designed to operate in a corporate environment, supporting complex business processes and organizational workflows.

Multi-User

Supports thousands of concurrent users across various departments globally without crashing.

Massive Data

Handles petabytes of data, processing high-volume transactions with complex persistence needs.

Secure & Reliable

Ensures 99.999% uptime, strict fault tolerance, role-based security, and data integrity.

Integrated

Seamlessly connects with databases, external APIs or legacy systems.

Key Insight for Students: Enterprise applications are not just larger high volume of code, they are built with strict software architecture and design patterns to allow developer teams to maintain them for decades.

Multi-Layer Architecture of Enterprise Systems

To keep code maintainable and testable, enterprise applications divide responsibilities across distinct main architectural layers

1. Presentation Layer

User Interface (UI / Web)

- Presents data to end-users
- Handles user input and user request validation
- Web pages and Mobile interfaces

2. Application Layer

Core Processing Engine

- Executes organizational rules
- Processes business logics and workflows
- Evaluates conditions and computations

3. Data Layer

Persistence Layer

- Communicates with databases
- Executes database queries
- Manages data transactions and entities

Quick Question: If a bank application calculates interest rates for a mortgage, which layer handles that code?

Real-World Enterprise Application Domains

Everyday Enterprise Software Examples

- **Banking & Financial Systems:** Core transaction engines, fraud detection, international wire transfers.
- **E-Commerce Platforms:** Inventory tracking, payment gateway processing, order fulfillment.
- **Hospital Management Systems:** Electronic health records, appointment scheduling, billing integration.
- **Enterprise Resource Planning (ERP):** Systems coordinating logistics, HR, and supply chain.
- **Customer Relationship Management (CRM):** Sales pipeline automation, customer tracking (e.g., Salesforce).

Where Does Java & Spring Fit In?

Java is the #1 language for backend enterprise development due to its stability, security, and mature ecosystem.

Spring is a framework designed to simplify the development of Java based enterprise applications.

</> **Industry Fact:** Over 85% of worldwide web-based Java enterprise services are powered by the **Spring Framework**!

The Spring Framework Ecosystem

Spring is not just a single framework. It is a comprehensive ecosystem of modular tools designed for modern software development.

🚀 Spring Boot

The standard way to build Spring based enterprise applications quickly. Offers auto-configuration, embedded servers (Tomcat), and rapid startup.

Spring Data

Simplifies database access across SQL & NoSQL databases.

Spring Security

Handles authentication, authorization.

Introduction to Spring Boot

Spring Boot

Enterprise Standard

Used by major banks, Netflix, Amazon - the industry chose Spring for a reason.

Embedded Server

Tomcat is built in. No separate install. Run the app like any Java program.

No XML Config

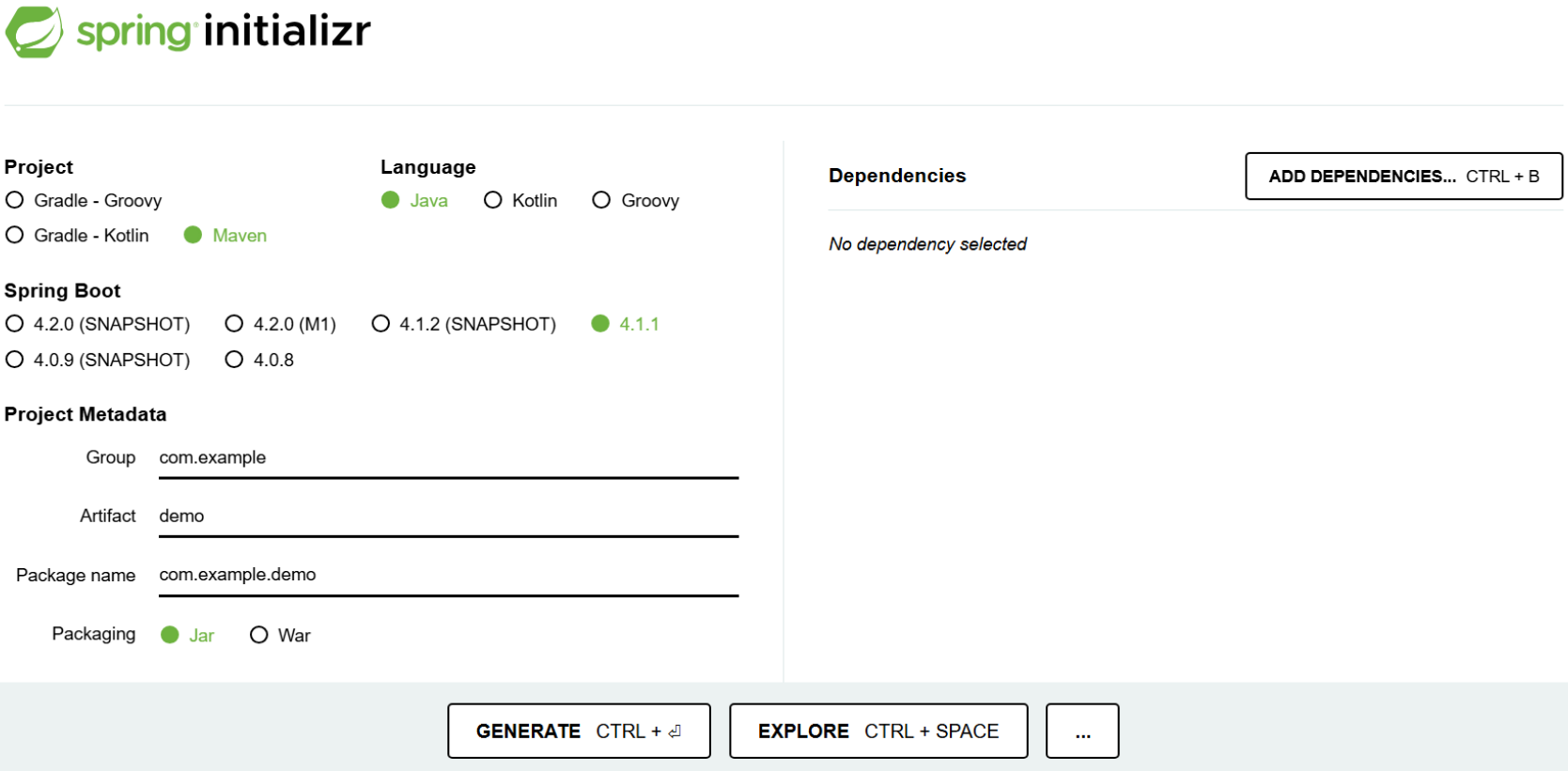
Spring Boot auto-configures almost everything. You write Java, not boilerplate XML.

Convention over Configuration

Sensible defaults out of the box. Override only what you need.

spring.io · Currently on Spring Boot 4.x · Requires Java 17+

Spring Initializr Walkthrough



Live demo - follow along at start.spring.io

Start.spring.io – Step by Step

01 **Open start.spring.io in the browser**
Everyone open this URL now - follow along.

02 **Project: Maven · Language: Java · Spring Boot: 4.x**
Maven manages our dependencies. We'll see pom.xml.

03 **Group: uk.ac.westminster · Artifact: products-api**
This becomes your Java package name.

04 **Packaging: Jar · Java: 17 or 21 or 25 or 26**
Jar = self-contained runnable. Java 17 = Long Term Support.

05 **Add dependency: Spring Web**
This one dependency adds an HTTP server to your app.

06 **Click Generate → unzip → open in IntelliJ**
File > Open, then wait for Maven to download dependencies. (If does not work check Version in pom.xml file)

You Just Started a Web Server

ProductsApiApplication - Running

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
 Started ProductsApiApplication in 1.847 seconds

 Tomcat started on port 8080 (http)
```



This is the first milestone.

An embedded Tomcat HTTP server is now listening on port 8080. Open a browser and go to http://localhost:8080 - you'll see a Whitelabel Error page. That means Spring is running but has no endpoint yet.

Understanding pom.xml file

```
<!-- pom.xml — your project's dependency manager -->
<project>
  <groupId>uk.ac.westminster</groupId>
  <artifactId>products-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <dependencies>
    <!--Spring Web = embedded Tomcat + REST support-->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>
</project>
```

groupId / artifactId

Like a unique address for your project on Maven Central.

dependencies block

Everything inside here gets downloaded and added to your project.

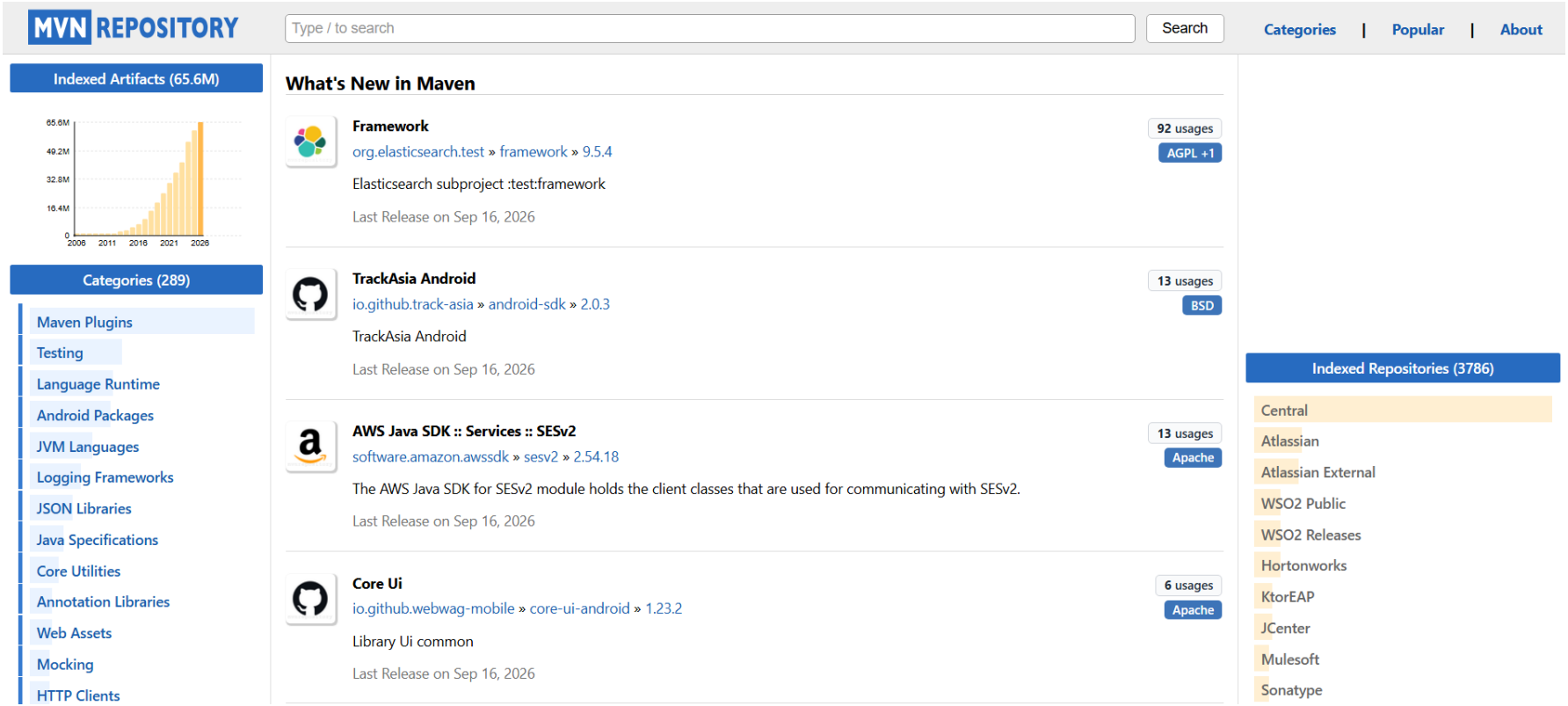
starter-web

A curated bundle: Spring MVC + Jackson JSON + Tomcat server.

No version needed

Spring Boot manages compatible versions automatically.

Manual Spring Boot Setup using MVN Repository



Manually initializing and configuring a Spring Boot application using standard Maven dependencies.

Setup Roadmap – Step by Step

01
Create a Maven Project
Initialize standard Maven directory structure in IntelliJ IDEA

02
Create Dependency Tree in pom.xml
Configure pom file and set up XML section.

03
Add Dependencies from MVN Repository
Include spring-boot-starter, spring-boot-starter-web, & spring-boot-starter-test.

04
Create AppInitializer Class
Write entry class with @SpringBootApplication annotation and main method.

05
Create application.properties file
Configure application settings in src/main/resources directory.

06
Run the Application
Execute main method to start embedded Tomcat server.

Maven Dependencies

Dependency Artifact	Purpose & Functionality	Key Features Included
spring-boot-starter	Core Spring Boot starter dependency	Auto-configuration support & default-configuration support.
spring-boot-starter-web	Web & REST endpoint development support	Embedded Apache Tomcat server, Spring MVC, Jackson JSON serializer.
spring-boot-starter-test	Testing library for unit testing	JUnit 5, Mockito, AssertJ, and Spring Test context support.

Create AppInitializer Class

```java
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AppInitializer    {

    public static void main(String[] args) {
        SpringApplication.run(AppInitializer.class , args);
    }
}
```

Key Components Breakdown

- **@SpringBootApplication:** Combines @SpringBootConfiguration, @EnableAutoConfiguration, and @ComponentScan.
- **Main Method:** Serves as the Java application entry point.
- **SpringApplication.run():** Launches the Spring context, performs component scanning, and starts embedded Tomcat server.

Configure Application Properties

Directory Location

Create application.properties in the standard resources directory:

Common Properties

```
# Server Port Configuration
server.port=8080

# Application Name
spring.application.name=products-api

# Logging Level
logging.level.org.springframework=INFO
```



Your First REST Endpoint

@RestController & @GetMapping live

What is a REST Endpoint?



HTTP Verb

GET reads data. POST creates. PUT updates. DELETE removes.

Path

/hello is the URL path. Spring matches this to your method.

Response

Your method's return value becomes the HTTP response body.

Status

200 OK = success. Spring sets this automatically for you.

LIVE CODE

HelloController - Type This Now

```
package uk.ac.westminster.productsapi;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import java.time.LocalDate;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot!";
    }

    @GetMapping("/status")
    public String status() {
        return "API running - " + LocalDate.now().toString();
    }
}
```

What each part does:

`@RestController`
Spring finds and manages this class

`@GetMapping`
Maps GET /hello → this method

`public String`
Method is accessible, returns text

`return …`
This text IS the HTTP response body

The Key Annotations

`@RestController`	Marks the class as a Spring component that handles HTTP requests. Spring detects it automatically at startup (component scanning). Every method can return a response body.
`@GetMapping`	Maps a URL path to a method. @GetMapping("/hello") means: when a GET request arrives at /hello, run this method. The return value becomes the response.
`@SpringBootApplication`	On your main class. Activates component scanning, auto-configuration, and the embedded server. The @RestController above is discovered because of this.

Test It in the Browser

1. Save HelloController.java
2. Stop the app (red square) → Restart (green play)
3. Open browser → navigate to localhost:8080/hello
4. You should see: Hello from Spring Boot!

If I add @GetMapping("/goodbye") returning "Goodbye!", what URL will respond to it?

07

Swagger UI

Making Your API Visible

One dependency transforms the developer experience



The Problem and the Fix

Without Swagger UI

localhost:8080

Whitelabel Error Page

Status=404

There was an unexpected error.

😟 **You don't know if their API is working**



With Swagger UI

localhost:8080/swagger-ui.html

GET /hello

GET /status

[Try it out] [Execute]

✅ **Visual, interactive, auto-generated**

LIVE CODE

Adding Swagger UI - One Dependency

```
<!-- Add this inside <dependencies> in pom.xml -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.3.0</version>
</dependency>

// ── That is ALL the code you need. ──
// 1. Save pom.xml
// 2. Maven auto-imports the library
// 3. Restart the application
// 4. Open: http://localhost:8080/swagger-ui.html
//
// Every @RestController endpoint appears automatically.
// No extra annotations, no extra configuration.
```

What to look for in Swagger UI

Endpoint List

Every @GetMapping / @PostMapping endpoint is listed automatically. Add a new endpoint, restart, refresh - it appears.

Try it out

Click 'Try it out' → 'Execute' to send a real HTTP request to your running app.

Response section

See the Response Body, HTTP Status Code (200 OK), and Response Headers. Everything you need to debug.

Industry standard

Swagger / OpenAPI is used professionally. Learning it now gives you vocabulary for job interviews and industry practice.

Version Control from Day One

Git + GitHub



Why Version Control from Lecture 1?

Requirement

The coursework explicitly requires VCS evidence. A repo with one commit on submission day demonstrates nothing about your development process.

Professional Habit

In industry, you commit early, commit often, and write meaningful messages. We build that habit from day one.

Safety Net

Git lets you experiment freely — if you break something, you can always roll back to the last working commit.

Coursework Evidence

Your commit history tells the story of your development. Assessors can see what you built and when. 11 weeks of commits is compelling.

```
# Step 1 — Initialize a git repository (run once per project)
git init

# Step 2 — Stage everything
git add .

# Step 3 — Commit with a MEANINGFUL message
git commit -m "Tutorial 1: Spring Boot project created, /hello endpoint
working"

# Step 4 — Connect to GitHub (run once)
git remote add origin https://github.com/[username]/products-api.git
git branch -M main

# Step 5 — Push to GitHub
git push -u origin main

# After the first push, subsequent pushes are just:
git push
```

Writing Good Commit Messages

❌ Poor Commit Messages

```
"fix"
```

```
"asdf"
```

```
"done"
```

```
"changes"
```

```
"update 3"
```

✓ Good Commit Messages

```
Lecture 1: Spring Boot project created
```

```
Lecture 2: Product class with encapsulation
```

```
Lecture 3: CRUD endpoints added to service
```

```
Lecture 4: Inheritance hierarchy implemented
```

```
fix: null pointer in getProductById
```

A commit message is a message to your future self.

Verify: Files on GitHub

github.com / [your-username] / products-api	
src/	source code
pom.xml	Maven configuration
.gitignore	tells Git to ignore target/ folder
README.md	optional — add one!

Lecture Summary & Key Takeaways

Multi-Layer Architecture

Enterprise apps separate concerns into Presentation, Application (business logic), and Data layers.

Spring Boot Power

Spring framework with auto-configuration, embedded Tomcat server, and production-ready defaults.

REST API Building

Annotations like @RestController and @GetMapping make creating scalable HTTP web services clean and intuitive.

Interactive Tooling

Dependencies like springdoc-openapi generate live, interactive Swagger API documentation effortlessly.

Student Knowledge Check

1. Which layer in a 3-tier architecture is responsible for evaluating business rules & workflows?

2. Which annotation informs Spring to detect a class and handle incoming HTTP REST requests?

3. What URL path opens the interactive Swagger UI interface once springdoc-openapi is added?

References

- **Spring Boot Reference Documentation (Official Spring Docs)**
  - **Coverage:** Complete guide on Spring Boot concepts, starter dependencies (spring-boot-starter-web), auto-configuration, and running embedded Tomcat web servers.
  - **URL:** docs.spring.io/spring-boot/docs/current/reference/html

- **Spring Initializr Guide & Tutorial (Spring.io)**
  - **Coverage:** Step-by-step documentation on bootstrapping new projects via start.spring.io, configuring Maven build metadata (pom.xml), and selecting dependencies.
  - **URL:** https://spring.io/guides/gs/spring-boot

- **Building a REST Web Service Guide (Spring Official Guides)**
  - **Coverage:** In-depth breakdown of core REST annotations including @RestController, @GetMapping, path mapping, and HTTP request-response flows.
  - **URL:** spring.io/guides/gs/rest-service

- **Springdoc OpenAPI / Swagger UI Official Documentation**
  - **Coverage:** Setup guide for adding springdoc-openapi-starter-webmvc-ui to Maven projects, enabling interactive API testing at /swagger-ui.html.
  - **URL:** springdoc.org